NVDS Over UART Application Note

Revision History

Date	Version	Description
September 30, 2021	0.50	Initial version created.

CONFIDENTIAL

©2021, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders. This preliminary document is subject to change without notice.

Table of Contents

1 Overview 3

2 UART Bootloader 4
- 2.1 SRAM Memory Overview 5
- 2.2 UART Bootloader Execution Options 6
  - 2.2.1 Execution from Flash 6
  - 2.2.2 Execution from SRAM Loaded from the OTP 6
- 2.3 Make Command Examples 7

3 Flash Over UART 8
- 3.1 UART Flash Requests and Response Format 8
  - 3.1.1 UART Flash Identify Format 9
  - 3.1.2 UART Flash Read Format 9
  - 3.1.3 UART Flash Write Format 9
  - 3.1.4 UART Flash Erase Format 10
- 3.2 Make Command Example 10

4 NVDS Message Charts 11
- 4.1 Read a NVDS Tag 11
- 4.2 Write a NVDS Tag 12

5 Quick Start 13

List of Figures

Figure 1 - Flash Over UART Block Diagram
Figure 2 - Flash over UART with UART Bootloader
Figure 3 - End-of-SYSRAM Organization
Figure 4 - Execution from Flash
Figure 5 - Execution from SRAM with Code Downloaded from OTP
Figure 6 - Flash Operations Over UART
Figure 7 - NVDS Read Message Flow Chart
Figure 8 - NVDS Write Message Flow Chart
Figure 9 - Testbed Overview
Figure 10 - Hardware Settings

CONFIDENTIAL

1 Overview

The Non Volatile Data Storage (NVDS) over UART feature, also referred to as "Flash over UART", is mainly for Atmosic CSP package or RAM based applications. This is used when the Atmosic device is connected to an external MCU through a UART interface and the flash storage is on the external MCU side. When the user wants to store some application records or NVDS tags, it will involve several flash operations. Since the flash storage is on the external MCU side, these operations should be redirected to the MCU through the UART. This feature can be enabled by specifying the UART_FLASH make option and a UART flash driver will be included in the RAM code as shown in Figure 1. In running state, if the external MCU gets the flash operation requests from the firmware, it should handle those requests and respond to them with specified data formats. See section 3 Flash Over UART section for more information about the UART flash driver.

CONFIDENTIAL

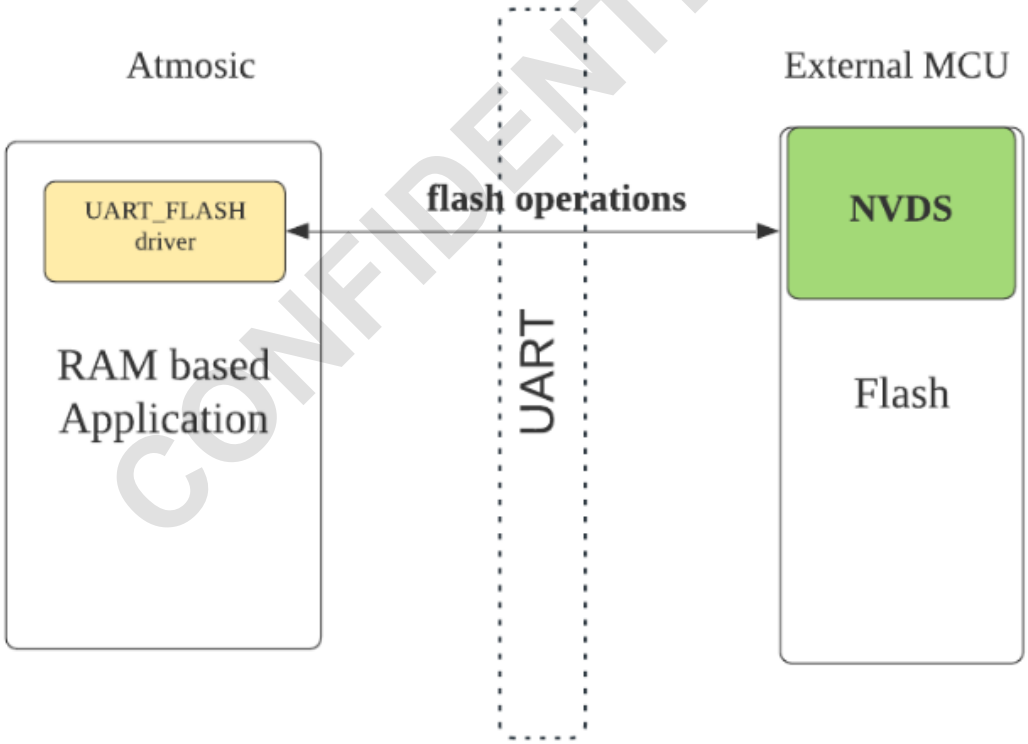


Figure 1 - Flash Over UART Block Diagram

In addition, a RAM based application is usually used with a UART bootloader. Figure 2 shows the workflow of the application which enables this feature. In the beginning, the Atmosic device boots up with a builded-in UART bootloader application in the one time programmable (OTP) or non-volatile memory. And then the external MCU will download the RAM based application binary file into the RAM which is inside the Atmosic chip through the UART bootloader by the XMODEM

protocol. After the binary file is fully downloaded into the RAM, the program counter will jump to the entry of the RAM based application and start to run the application. If the application has to store the NVDS tags or other records in running state, these flash operation requests will be redirected to the external MCU by the UART flash driver and the driver will also handle the responses from the MCU. Refer to section 2 UART Bootloader section for more details about the UART bootloader.

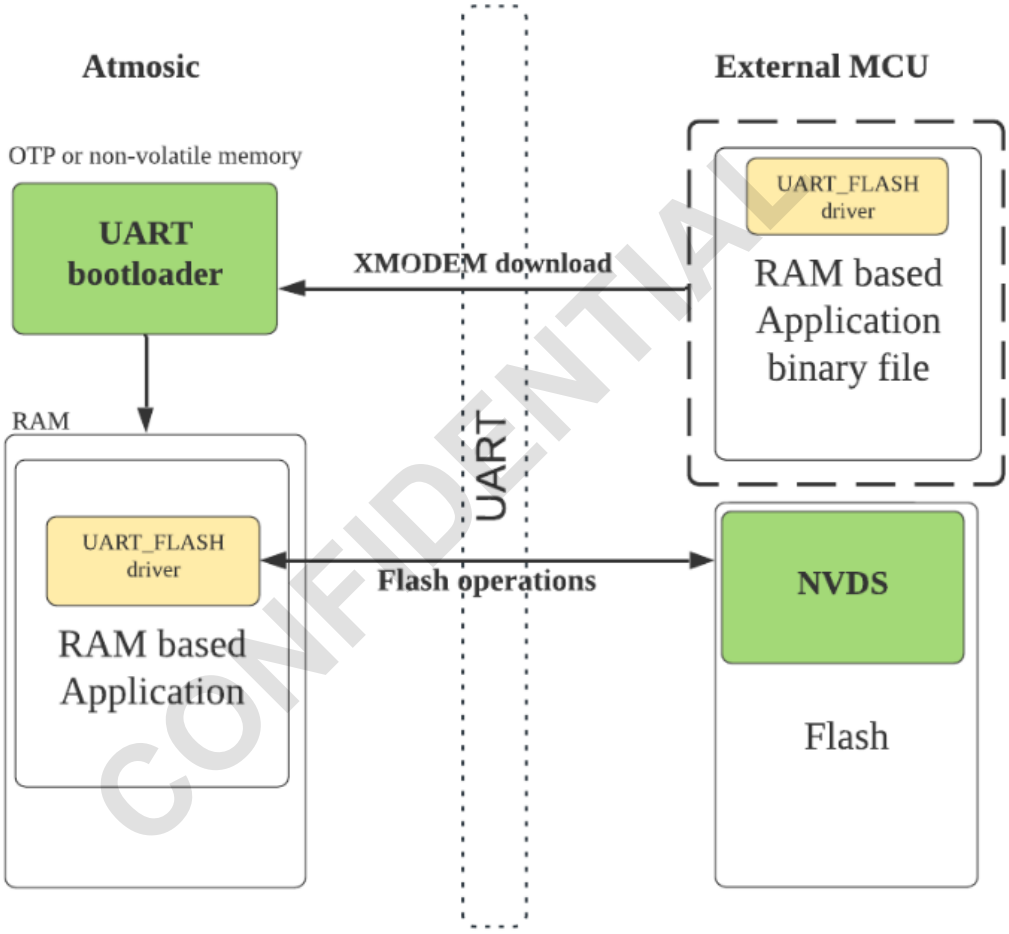


Figure 2 - Flash over UART with UART Bootloader

2 UART Bootloader

To load the RAM code to the SYSRAM, a UART bootloader can be used. The UART bootloader is responsible for loading the RAM code from the UART and jumping to the application entry after the RAM code is fully loaded. To ensure the correctness of the RAM code, the XMODEM protocol is used in the transmission through UART. The details on the implementation of the XMODEM protocol, please refer to the source file “xmodem.c” under “UART_bootloader/client/”. The UART bootloader can be configured to operate on either UART0 or UART1 through the compile time

switch, CFG_UBOOT_UART0. The default serial port parameters are 115200 baud, 8 bits, no parity, 1 stop bit, no RTS/CTS flow control. The start address for loading the RAM code firmware can be configured by the makefile LOAD_ADDR option. There are two ways to run a UART bootloader, loading from OTP or executing from flash, see section 2.2 UART Bootloader Execution Options. To build a RAM code binary for the UART bootloader, the makefile RUN_IN_RAM option is necessary in your application. The makefile URAM_START option and the URAM_SIZE option should also be used in the make command. The URAM_START option has to be specified with the same address assigned in the LOAD_ADDR option and the URAM_SIZE option is used to specify the maximum URAM size of this application.

2.1 SRAM Memory Overview

Figure 3 shows the end-of-SYSRAM organization. The maximum size of the user application can be 56 KB leaving 8 KB for heap, bootloader and stack. The address from 0x20009000 to 0x20009800 is for using the flash to run the UART bootloader and the address from 0x20008000 to 0x20008a00 is for using the OTP. Since the start address is configurable, the application needs to be cognizant that increasing application memory has a direct impact on reduced heap/stack. Therefore applications need to consider dynamic memory allocation along with stack usage before expanding application size.

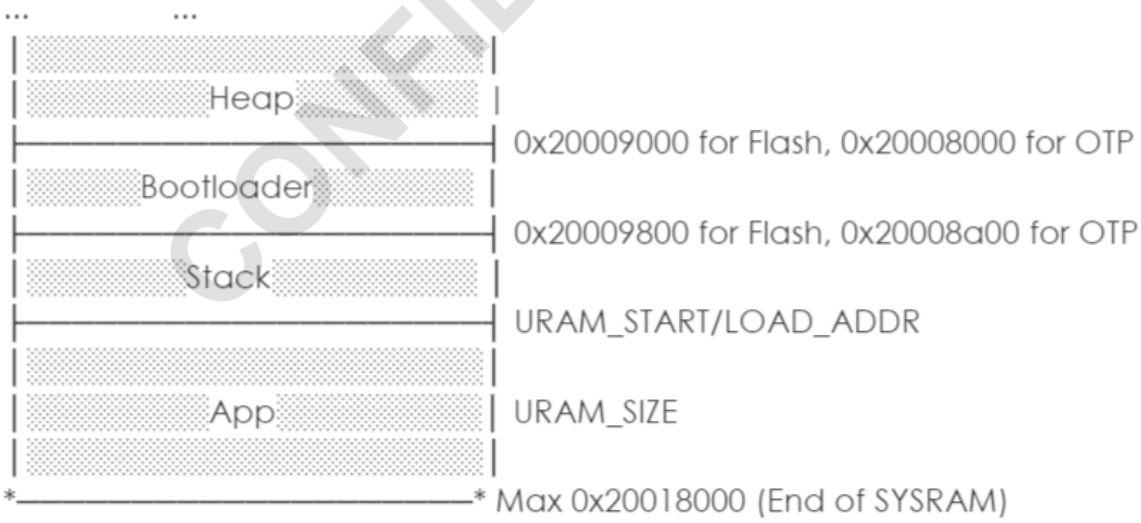


Figure 3 - End-of-SYSRAM Organization

CONFIDENTIAL

2.2 UART Bootloader Execution Options

The UART bootloader can be running from flash storage or running on the SRAM loaded from the OTP. For early development and debugging, it will be more convenient to use the flash option. The SRAM option is predominantly used for the CSP package which has no internal flash storage.

2.2.1 Execution from Flash

For testing or debugging purposes, the UART bootloader can be executed from the flash. The code section of the UART bootloader is located in the flash and only a small portion of the SRAM is used for the static variables in the UART bootloader. The RAM based firmware will go through the UART bootloader and be downloaded into the SRAM.

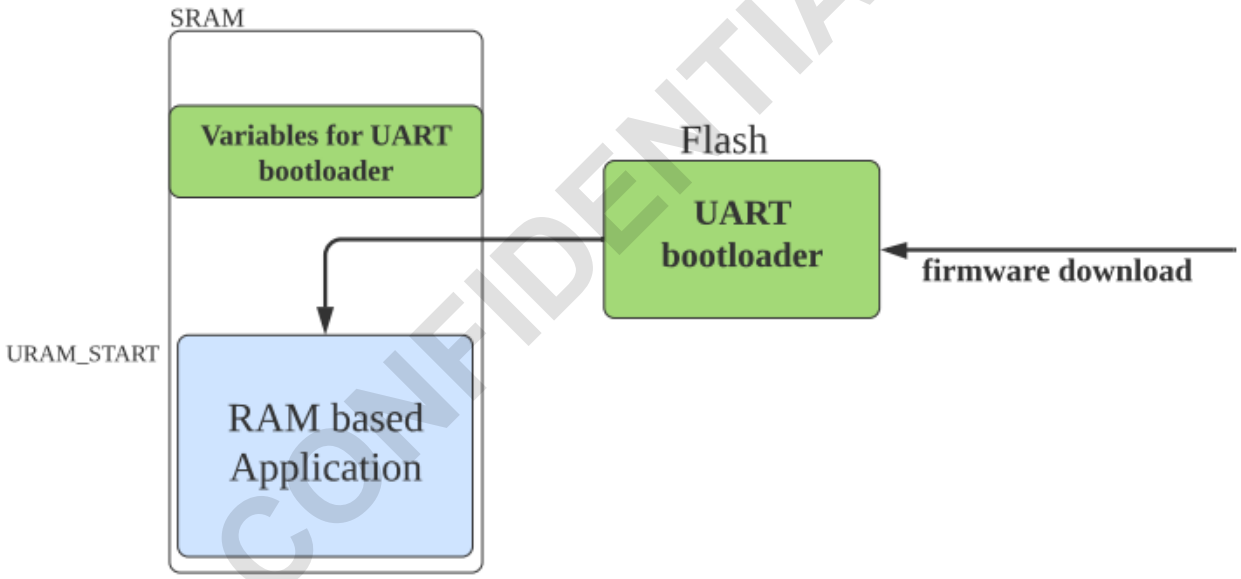


Figure 4 - Execution from Flash

2.2.2 Execution from SRAM Loaded from the OTP

When the application is planning to use the chip without the flash storage such as the CSP package, another option is to put the UART bootloader into the OTP. The binary file of the UART bootloader can be burned into the OTP with an application tag. When the ROM code finds the application tag, it will copy all of the binary content into the SRAM and start to execute the binary application. After these operations are done, the UART bootloader will be running on the SRAM and the firmware downloading process will also go through the RAM based UART bootloader. Finally, after the firmware is downloaded, the program counter will jump to the application entry to execute the user RAM based user application.

CONFIDENTIAL

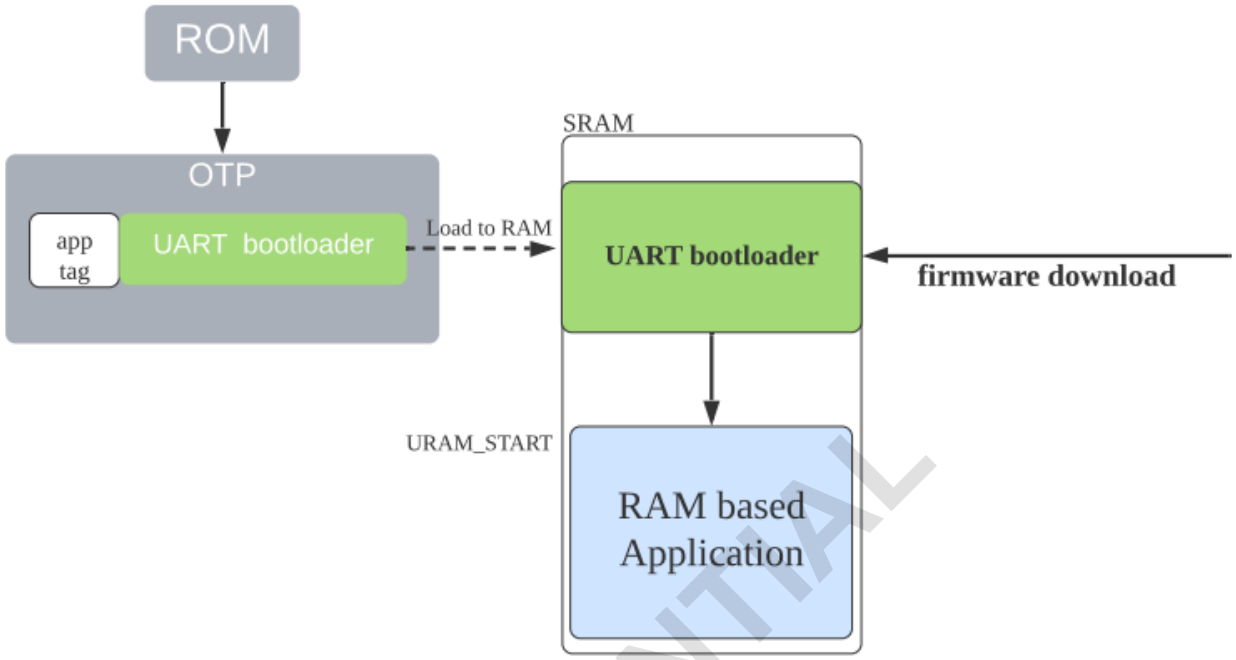


Figure 5 - Execution from SRAM with Code Downloaded from OTP

2.3 Make Command Examples

1. Build UART bootloader:
   make clean
   make LOAD_ADDR:=0x2000c000 all

2. Build a RAM application, for example: under /BLE_scan/
   make clean
   make RUN_IN_RAM:=1 URAM_START:=0x2000c000 URAM_SIZE:=0xc000 all

3. Load UART bootloader form OTP:
   make pull_otp_nvds
   make rebuild_otp_nvds
   make push_otp_nvds

4. Execute from flash:
   make LOAD_ADDR:=0x2000c000 run_all

CONFIDENTIAL

3 Flash Over UART

To support NVDS and flash read/write on a RAM based application, a UART flash driver can be used by specifying the UART_FLASH make option. 'UART_FLASH=0' is for UART0 and 'UART_FLASH=1' is for UART1. After the driver is loaded to the firmware, four flash APIs, which are flash_identify, flash_read, flash_write and flash_erase, will be replaced by the driver and those requests will be redirected to external MCU through UART with specified data formats. The external MCU should handle those requests and respond to them with specified formats too. Also, when the UART flash driver is included in the firmware, the start offset of the NVDS will be set to 0x00000000 and the default maximum NVDS region size is 32 KB. The maximum NVDS region size can also be configured by the makefile UART_FLASH_NVDS_SIZE option and this is a boundary for the firmware to read/write NVDS related data on the external MCU.

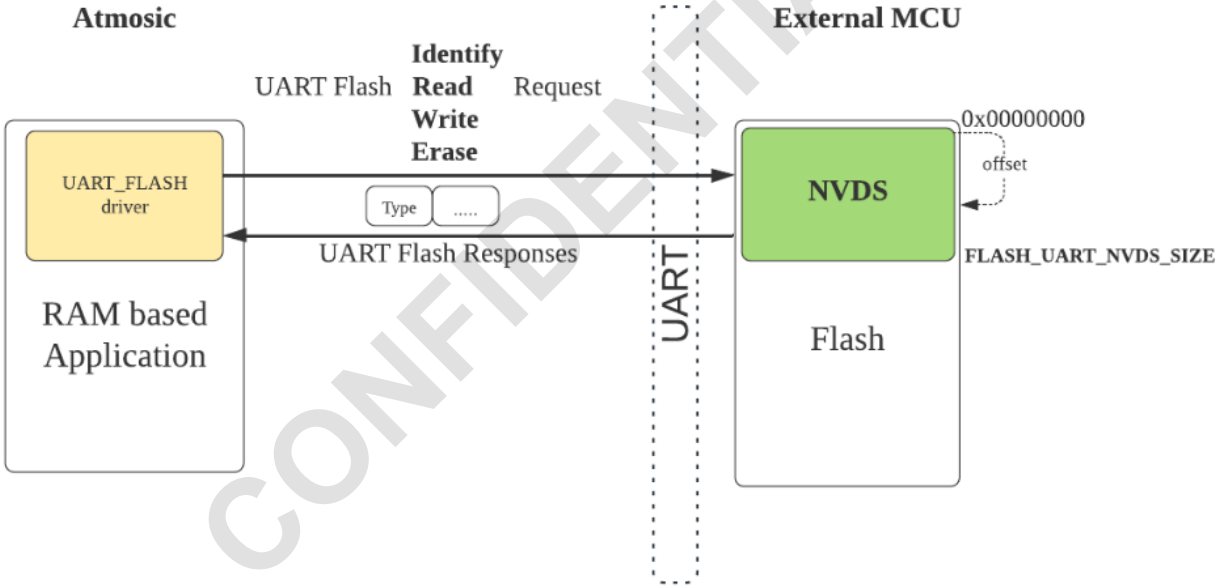


Figure 6 - Flash Operations Over UART

3.1 UART Flash Requests and Response Format

When the UART flash driver is included in the firmware, the flash operation requests will be redirected to the external MCU through the UART interface. The external MCU should parse those requests and respond to them with specified data format. There are four flash APIs replaced by the driver. The users should be aware that the requests will go through the UART when they are calling these APIs. Also, on the other side, the external MCU has to implement the handlers for those requests by the following formats. The implementation details can be found in the source file “host_nvds.c” under “ /UART_bootloader/client/ ”.

CONFIDENTIAL

3.1.1 UART Flash Identify Format

The flash_identify API is used to read the flash device id. A UART_flash_identify request will be sent through the UART while calling this API. After the request is received by the MCU, the MCU should send a UART_flash_identify response back to the peer. In the UART_flash_identify response, the flash id field should be a non-zero value and the status field should be set to zero if the operation has no error.

```
uint8_t flash_identify(uint8_t* id, void (*callback)(void));
```

UART_flash_identify request

Type: **0** (1 byte)

UART_flash_identify response

Type: **0** (1 byte)	Status (1byte)	Flash id (1byte)

NOTE: Status (0:NO_ERROR 1:FAILED) Flash_id (non-zero)

3.1.2 UART Flash Read Format

Flash_read API is used to read a portion of the flash memory. The UART_flash_read request carries the starting offset from the beginning of the flash device and the size of the portion of flash memory to read. The MCU should send the data content of the part of flash memory back to the peer in the UART_flash_read response. When the driver gets the data, it will copy them to the buffer.

```
uint8_t flash_read(uint8_t flash_type, uint32_t offset, uint32_t length,
uint8_t *buffer, void (*callback)(void));
```

UART_flash_read request

Type: **1** (1 byte)	Offset (4 bytes)	Length (4 bytes)

UART_flash_read response

Type: **1** (1 byte)	Status (1 byte)	Length (4 bytes)	Data ((length) bytes)

NOTE: Status (0:NO_ERROR 1:FAILED)

3.1.3 UART Flash Write Format

Flash_write API is used to write a portion of the flash memory. The UART_flash_write request carries the starting offset from the beginning of the flash device and the size of the portion of flash to write. Also, the data in the buffer will also be appended in the tail of the request too. When the

CONFIDENTIAL

MCU receives the request, it should write the data to the flash and reply the status of the operation to the peer in the UART_flash_write response.

```
uint8_t flash_write(uint8_t flash_type, uint32_t offset, uint32_t length,
const uint8_t *buffer, void (*callback)(void));
```

UART_flash_write request

Type: **2** (1 byte)	Offset (4 bytes)	Length (4 bytes)	Data ((length) bytes)

UART_flash_write response

Type: **2** (1 byte)	Status (1byte)

NOTE: Status (0:NO_ERROR 1:FAILED)

3.1.4 UART Flash Erase Format

Flash_erase API is used to erase a portion of the flash memory. The UART_flash_erase request carries the starting offset from the beginning of the flash device and the size of the portion of flash to erase. When the MCU receives the request, it should erase the portion of the flash memory and reply the status of the operation to the peer in the UART_flash_erase response.

```
uint8_t flash_erase(uint8_t flash_type, uint32_t offset, uint32_t size,
void (*callback)(void));
```

UART_flash_erase request

Type: **3** (1 byte)	Offset (4 bytes)	Length (4 bytes)

UART_flash_erase response

Type: **3** (1 byte)	Status (1byte)

NOTE: Status (0:NO_ERROR 1:FAILED)

3.2 Make Command Example

Pick an example and type the following make command, for example, BLE_adv.

Using UART0:
make RUN_IN_RAM:=1 URAM_START:=0x2000c000 URAM_SIZE:=0xc000 UART_FLASH:=0 UART_FLASH_NVDS_SIZE=0x400 all

CONFIDENTIAL

Using UART1:
make RUN_IN_RAM:=1 URAM_START:=0x2000c000 URAM_SIZE:=0xc000 UART_FLASH:=1 UART_FLASH_NVDS_SIZE=0x400 all

4 NVDS Message Charts

NVDS operations are all based on the flash APIs. When the **UART_FLASH** make option is used, the NVDS read and write operation will also go through the UART flash driver. The start offset of the NVDS is set to 0x00000000 by default and the boundary of the NVDS region can be configured by **UART_FLASH_NVDS_SIZE**. There are two common APIs for the user to read and write their NVDS tag. There are **nvds_get** and **nvds_put**. Relationships between the APIs and the UART flash driver are described in the message flow charts. See Figure 7 and Figure 8.

4.1 Read a NVDS Tag

```
uint8_t nvds_get(uint8_t tag, nvds_tag_len_t * lengthPtr, uint8_t *buf);
```

To read a NVDS tag, **nvds_get** API is called in the application and the usage won't be changed if the UART flash driver is used. Figure 7 shows the message flow chart for calling a nvds_get API. Flash read function will be invoked while calling the nvds_get API. And then the UART flash driver relays the flash read request to the external MCU by turning it into **UART_flash_read** request. The MCU should handle this request by reading the data from the NVDS region based on the offset and length. The data will be sent back to the driver with a **UART_flash_read** response. Finally the data will be copied to the buffer and return the status to the application.

CONFIDENTIAL

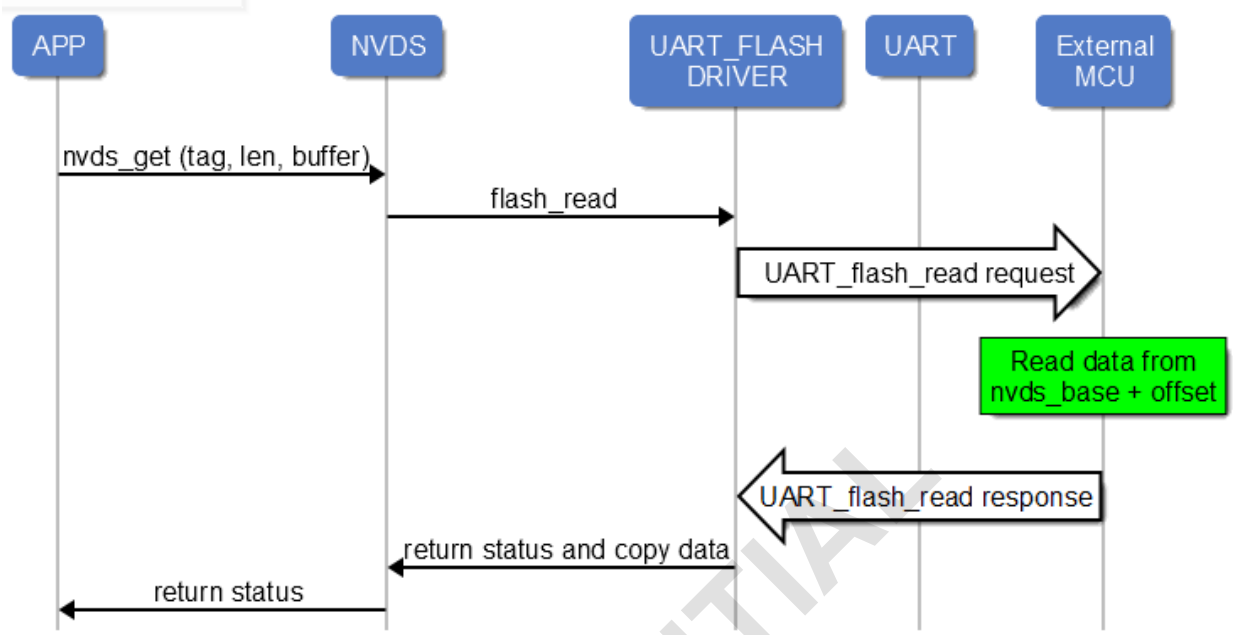


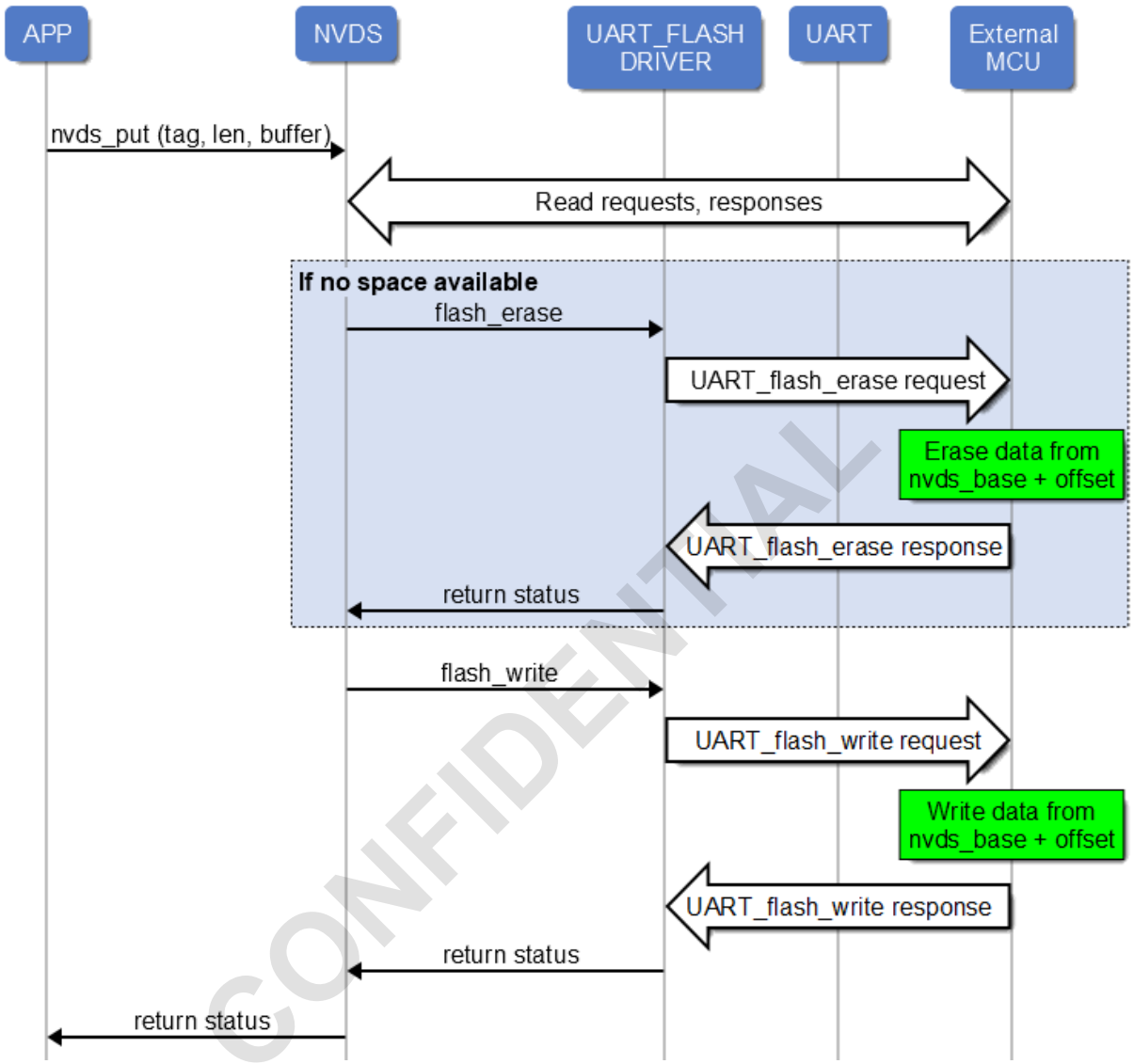
Figure 7 - NVDS Read Message Flow Chart

4.2 Write a NVDS Tag

```
uint8_t nvds_put(uint8_t tag, nvds_tag_len_t length, const uint8_t *buf);
```

To save a NVDS tag, we usually call a nvds_put API in the application and the usage is also the same as before if the UART flash driver is used. Figure 8 shows the message flow chart for saving a NVDS tag. To search the available space to write the NVDS tag, there might be several flash read requests before writing the NVDS record. If there are too many invalid tags in the NVDS region and the new NVDS tag can not be saved within the boundary, the NVDS firmware module will ask for a flash erase operation. The module will backup all of the valid tags and erase the whole NVDS region. After the erase operation has been completed, the valid tags will be rewritten to the NVDS region. If the available space is enough, the module will directly send a write request and it will be turned into UART_flash_write request to the external MCU. And then the MCU writes the data to the flash and responds the status back to the driver by a UART_flash_write response. In the end, the final status will return to the application and these are all of the operations that can be involved in a nvds_put API calling.

CONFIDENTIAL



CONFIDENTIAL

Figure 8 - NVDS Write Message Flow Chart

5 Quick Start

This is a quick start to demonstrate the NVDS over UART feature on the Atmosic Evaluation Kit (EVK). The evaluation board (EVB) is connected to a PC as shown in Figure 9. In this environment, UART0 is used for XMODEM firmware download and UART flash operations. UART1 is used for debug logs. Using the PC, build the FwUpload program which the source code is under '/UART_bootloader/client/' directory in the software development kit (SDK). The BLE_adv example is used as the RAM based application in this test. The FwUpload program will upload the RAM code binary file and NVDS binary file. On the Atmosic EVK, a UART bootloader will be running from the flash for downloading the RAM based firmware.

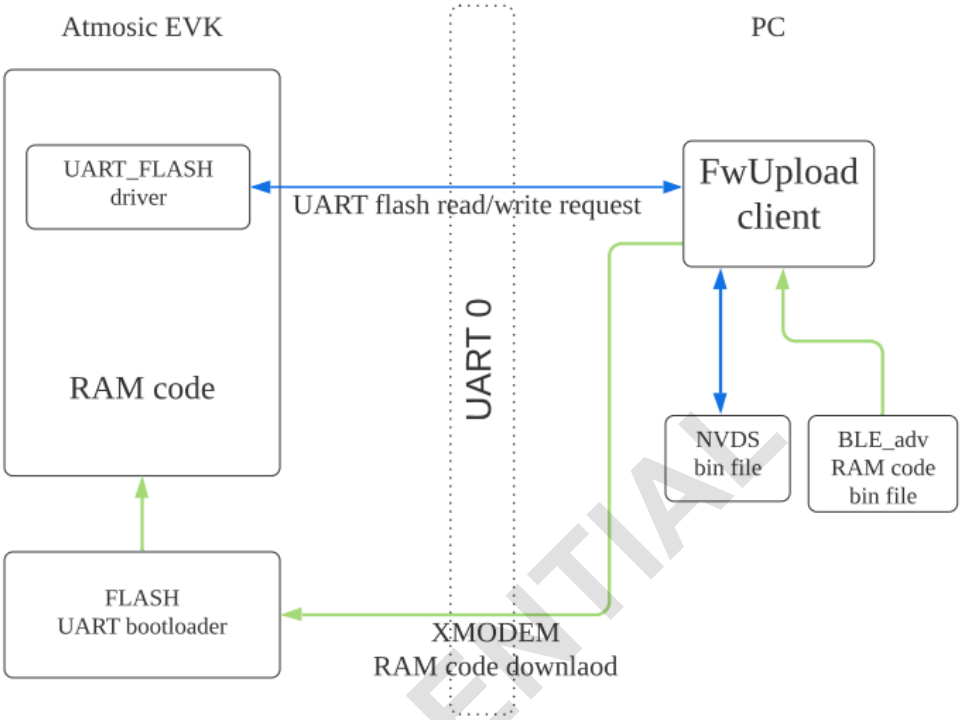


Figure 9 - Testbed Overview

The EVK contains an interface board, an Atmosic EVB and an antenna. Two USB ports are connected to the PC. USB0 is responsible for receiving and transmitting the data such as flash read/write requests from UART0. USB1 is for receiving debug logs from UART1. For more information, please refer to the EVK User Guide on the Atmosic Support Website

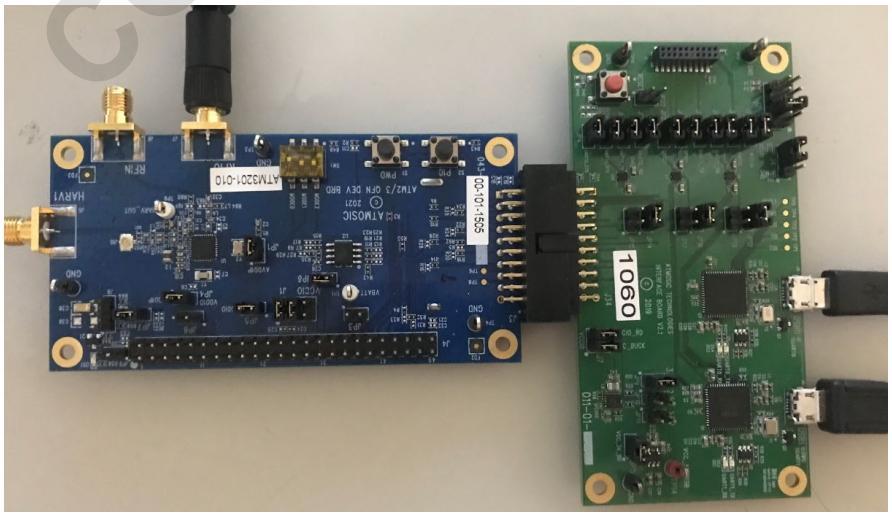


Figure 10 - Hardware Settings

1. To build and run UART bootloader in flash under /UART_bootloader/

CONFIDENTIAL

make clean

```
$ make clean
rm -f tag_data/2e-SLEEP_ALGO_DUR/sleep_algo.tds
rm -f tag_data/18-PROG_DELAY/prog_delay.tds
rm -f tag_data/b5-SYDNEY_TAG_WATCHDOG/wdog.tds
rm -f tag_data/*/*.bin
rm -f tag_data/*/*_pre_bond.*
```

make LOAD_ADDR:=0x2000c000 run_all

```
$ make LOAD_ADDR:=0x2000c000 run_all
mkdir -p tag_data/b5-SYDNEY_TAG_WATCHDOG
echo 00 b4 c4 04 > tag_data/b5-SYDNEY_TAG_WATCHDOG/wdog.tds
sed 's,#.*$,,' < tag_data/b5-SYDNEY_TAG_WATCHDOG/wdog.tds | tr -d '[
G/wdog.bin
mkdir -p tag_data/2e-SLEEP_ALGO_DUR
echo "F4 01"  > tag_data/2e-SLEEP_ALGO_DUR/sleep_algo.tds
sed 's,#.*$,,' < tag_data/2e-SLEEP_ALGO_DUR/sleep_algo.tds | tr -d '
eep_algo.bin
```

Logs from UART1

```
@00000026 ATM3201-x1x silicon: 5x5 pkg, External flash
@000000d3 SDK Version: 4.1.0
@0000012a APP Version: 0.0.0.9
5x5 EXT_FLASH: 4e 56 44 53 b5 06 04 00 b4 c4 04 2e 06 02 f4 01 ...
@00000298 xmodem_download_firmware(0x2000c000) sp:0x2000bfb8  00000026 ATM3201-x1x silicon: 5x5 pkg, External flash
```

2. To build BLE_adv example and flash NVDS binary files under /BLE_adv/

   make clean

   make RUN_IN_RAM:=1 URAM_START:=0x2000c000 URAM_SIZE:=0xc000 UART_FLASH:=0 UART_FLASH_NVDS_SIZE=0x400 all

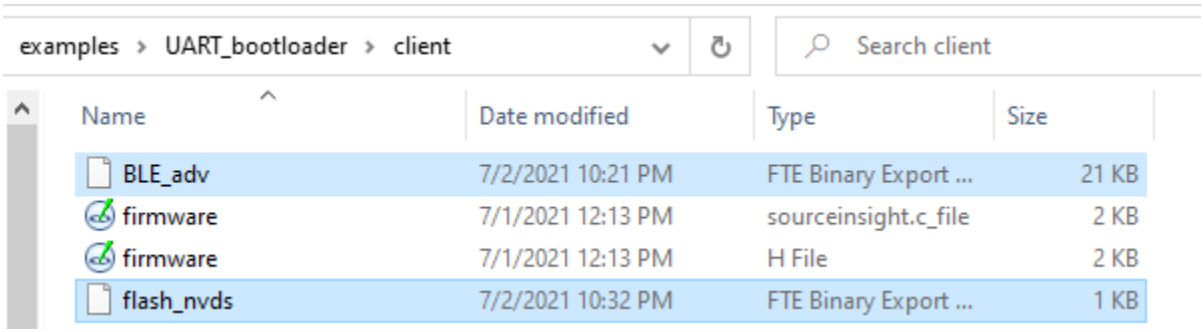
```
$ make RUN_IN_RAM:=1 URAM_START:=0x2000c000 URAM_SIZE:=0xc000 UART_FLASH:=0 UART_FLASH_NVDS_SIZE=0x400 all
/c/AtmosicSDK/4.1.0/tools/gcc-arm-none-eabi-8-2019-q3-update/bin/arm-none-eabi-as -mcpu=cortex-m0 -mthumb -mcpu
g  /c/AtmosicSDK/4.1.0/platform/atm2/ATM22xx-x1x/rom/user_startup_CMSDK_CM0.s -o user_startup_CMSDK_CM0.o
/c/AtmosicSDK/4.1.0/tools/gcc-arm-none-eabi-8-2019-q3-update/bin/arm-none-eabi-gcc -mcpu=cortex-m0 -Os -mthumb
 -fms-extensions -ffunction-sections -fdata-sections -fno-strict-aliasing -g3  -Wall -Wchar-subscripts -Wformat
gnedness -Wuninitialized -Winit-self -Wignored-qualifiers -Wswitch-default -Wswitch-enum -Wunused -Wundef -Wnul
e -Wsuggest-attribute=noreturn -Wdouble-promotion -Wwrite-strings -Wshadow -Wmissing-declarations -Wcast-qual -
rict-prototypes -Wold-style-declaration -Wold-style-definition -Wmissing-parameter-type -Wmissing-prototypes  -
ODULE__=\"user_init.c\"  -DCORTEX_M0 -DCFG_EMB -DCFG_HOST -DCFG_BLE -DCFG_AHITL -DCFG_NVDS -DCFG_DBG_MEM -DCFG_
CFG_DBG_NVDS -DCFG_DBG_STACK_PROF -DCFG_RF_SYDNEY -DCFG_ALLROLES -DCFG_ACT=6 -DCFG_BAL=3 -DCFG_CON=5 -DCFG_ISO
```

   make build_flash_nvds

```
$ make build_flash_nvds
sed 's,#.*$,,' < tag_data/06-APP_BLE_ACT_CRT_CMD/legacy_nc_s_100ms.tds | tr -d
LE_ACT_CRT_CMD/legacy_nc_s_100ms.bin
sed 's,#.*$,,' < tag_data/0b-APP_BLE_ADV_DATA/simple_beacon.tds | tr -d '[:spa
DATA/simple_beacon.bin
sed 's,#.*$,,' < tag_data/0c-APP_BLE_SCAN_RESP_DATA/simple_beacon.tds | tr -d
```

3. Put BLE_adv.bin and flash_nvds.bin from ‘ /BLE_adv/ ’ to ‘ /UART_bootloader/client/’

```
$ make clean
rm -f tag_data/2e-SLEEP_ALGO_DUR/sleep_algo.tds
rm -f tag_data/18-PROG_DELAY/prog_delay.tds
rm -f tag_data/b5-SYDNEY_TAG_WATCHDOG/wdog.tds
rm -f tag_data/*/*.bin
rm -f tag_data/*/*_pre_bond.*
```



4. Build fwupload.exe program under ‘ /UART_bootloader/client/ ’
   make

```
$ make
gcc -Wall -Werror -g    -c -o firmware.o firmware.c
gcc -Wall -Werror -g    -c -o main.o main.c
gcc -Wall -Werror -g    -c -o nvds_host.o nvds_host.c
gcc -Wall -Werror -g    -c -o uart.o uart.c
gcc -Wall -Werror -g    -c -o xmodem.o xmodem.c
gcc -o fwupload firmware.o main.o nvds_host.o uart.o xmodem.o -lm
```

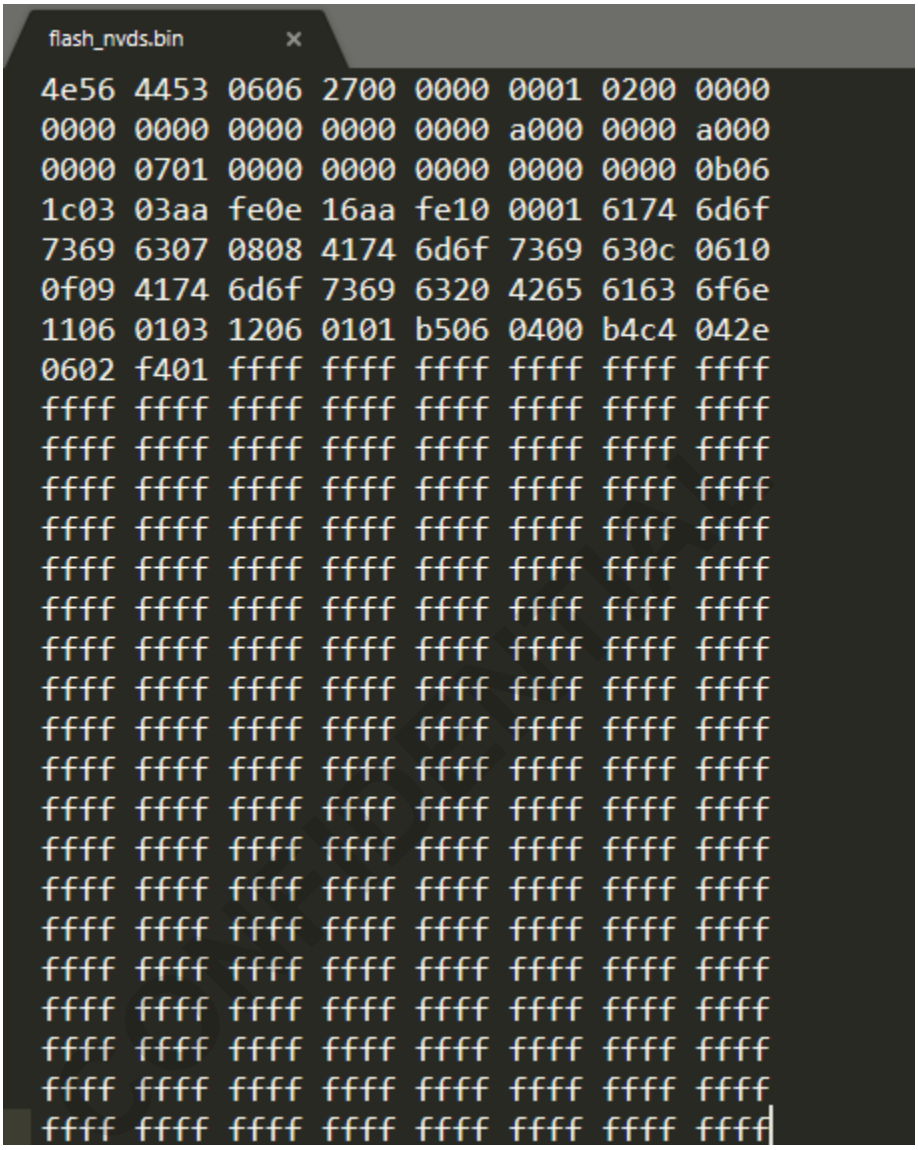
5. Run the program and specify the name of the two binary files. Also, needs serial port and NVDS region size as inputs under ‘ /UART_bootloader/client/ ’
   ./fwupload -f BLE_adv.bin -p <portname> -n flash_nvds.bin -s 0x400

```
$ ./fwupload -f BLE_adv.bin -p /dev/ttyS9 -n flash_nvds.bin -s 0x400
Filename: BLE_adv.bin, Portname: /dev/ttyS9, XMODEM block size: 128 bytes
Firmware upload ................................................................
................................................... successful
Init NVDS with maximum size 1024
NVDS init success, Running NVDS...
```

   Logs from UART1

```
5x5 EXT_FLASH: 4e 56 44 53 b5 06 04 00 b4 c4 04 2e 06 02 f4 01 ...
20000289 xmodem_download_firmware(0x2000c000) sp:0x2000bfb8
2000280d xmodem_download_firmware(0x20011200) sbrk:0x20009000
2000281d ATM3201-x1x silicon: 5x5 pkg, External flash
200028f6 SDK Version: 4.1.0
2000282e APP Version: 0.0.0.9
200028af UART: clk is 16000000, BAUDDIV is 139, baud is 115107
200028d8 [   BLE_adv][0]: user_main() done
200028e1 [   BLE_adv][0]: ble_adv_init: NVDS tag for adv timeout param not found. Using default
20002863 pseq_main_init() done
20002e44 [   atm_adv][0]: Advertising duration 0(in unit of 10ms) max_adv_evt 0 (timeout 0ms)
20002f92 [   BLE_adv][0]: adv_state = 2
20003041 [   BLE_adv][0]: adv_state = 4
200030ea [   BLE_adv][0]: adv_state = 6
20003199 [   atm_adv][0]: Adv0: ON (0)
20003231 [   BLE_adv][0]: adv_state = 9
```

6. The firmware is now running from RAM and all of the flash operations for NVDS will go through the UART interface until the port is disconnected. In addition, the whole NVDS region will dump to the original `flash_nvds`.bin and the user can reuse this file if needed.


flash_nvds.bin
4e56 4453 0606 2700 0000 0001 0200 0000
0000 0000 0000 0000 0000 a000 0000 a000
0000 0701 0000 0000 0000 0000 0000 0b06
1c03 03aa fe0e 16aa fe10 0001 6174 6d6f
7369 6307 0808 4174 6d6f 7369 630c 0610
0f09 4174 6d6f 7369 6320 4265 6163 6f6e
1106 0103 1206 0101 b506 0400 b4c4 042e
0602 f401 ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff
ffff ffff ffff ffff ffff ffff ffff ffff



CONFIDENTIAL

ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2021 by Atmosic Technologies. All rights reserved.

www.atmosic.com